UCS503 – Project Report

CampusLink

A Goal-Matching & Resource-Sharing Platform for College Campuses

"Find your people – beyond your friend circle."

Submitted to:

Ma'am Anushka
Computer Science and Engineering Department

Submitted by:

1024030968	Govind
1024030096	Ashish
[Roll No.]	Divya

Group: BE Third Year, CSE

Thapar Institute of Engineering and Technology, Patiala

Contents

Chapter 1

Introduction

1.1 Background and Context

On a large residential campus, day-to-day coordination between students still happens through fragmented, informal channels – batch WhatsApp groups, hostel groups and word of mouth. This works only when a student already knows the right people: if nobody in a friend circle is free for a gym session, or headed the same way for a cab, the plan simply does not happen, even though someone else on campus, statistically, probably wants the exact same thing. Sharing class notes suffers from the same limitation, and there is no lasting record of whether someone follows through on a plan once one is made.

CampusLink is proposed as a web-based goal-matching and resource-sharing platform for college campuses. It reframes an activity that needs another person as a *goal*: a student posts it, the goal appears on a public campus feed, interested students request to join, and once enough people are in, a small lobby forms and the group coordinates from there. The same post $\rightarrow$ request $\rightarrow$ fulfil loop is reused for a smaller, related use case – academic notes and resource sharing – where a responder reaches out privately instead of joining a group.

1.1.1 Problem Statement

Everyday coordination on campus has a few recurring pain points that CampusLink is designed to address:

- **Limited reach:** if a student does not personally know someone free for a given activity, the plan does not happen, even though a match likely exists elsewhere on campus.
- **No structure for stranger interaction:** there is no safe, trust-building layer for meeting someone unfamiliar, which matters more for things such as late-night rides or one-on-one meetups.
- **Inefficient notes and resource sharing:** asking in several groups for last semester's notes often gets no response, or a response too late to be useful.
- **No accountability:** students flake on plans with no record of it, so the next person has no way to judge reliability before committing time to them.

1.2 Project Objectives

1. **Objective 1:** Let verified students post and discover "goals" – activities that need a companion – through a public, filterable campus feed.

2. **Objective 2:** Provide a structured request-to-join and lobby-formation workflow, so a plan becomes a group only once an organizer approves the right people (or auto-accepts, by their own rule).
3. **Objective 3:** Build a trust and safety layer – reliability scoring, block/report, gender-preference filtering and anonymous posting – so interacting with a stranger feels workable.
4. **Objective 4:** Support real-time coordination through in-lobby chat, and a separate private chat for the notes/resource-sharing flow.
5. **Objective 5:** Record goal outcomes, no-shows and post-goal ratings so future users can judge an organizer's reliability before committing.
6. **Objective 6:** Offer simple, rule-based match suggestions using shared interests, branch and previously joined categories.
7. **Objective 7:** Design the platform as a stateless, horizontally scalable three-tier web application, deployable on free-tier hosting for a live demo.

These objectives describe the platform as scoped in the project proposal; Chapter 4 revisits which of them are realised in the current build and which remain future work.

Chapter 2

Methodology and Design

2.1 System Architecture

CampusLink follows a standard three-tier web architecture connecting a React frontend, a Node.js/Express backend API and a persistent database, together with a real-time layer used for lobby chat and live feed updates.

2.1.1 High-Level Design

- **Frontend:** a responsive React (Vite) application covering signup/login, the public feed with category and date filters, goal detail and posting screens, profile pages, and in-lobby chat.
- **Backend API:** authentication (including college-email OTP verification), goal management, the request/lobby state machine, and the trust-and-safety and matching endpoints, organised as a thin route $\rightarrow$ service layering.
- **Database:** users, goals, requests, lobbies, messages, ratings, reports, blocks and OTP tokens, as detailed in Section 2.3.
- **Real-time layer:** Socket.IO, used for live lobby updates and in-app chat so that neither depends on client-side polling.
- **Trust and safety services:** reliability scoring, no-show flagging, report/block handling, gender-preference filtering and anonymous posting, sitting alongside the core goal workflow rather than bolted on afterwards.

An important scoping note, carried through the rest of this chapter: the current codebase already serves authentication, profile and feed data end-to-end (Section 2.5.5 shows this from a live development session), while the lobby chat, matching-suggestion and several trust-and-safety pieces are present in the design – Figures 2.1–2.4 – but not yet wired into the running build. This report describes those as planned rather than implemented, and Section 4.3 lists them as concrete next steps.

2.1.2 Technology and Component Stack

- **Frontend** – React 18 + Vite, responsive layout, category-filtered feed and profile views.
- **Backend API** – Node.js + Express, REST routes for auth, goals, requests and reports.
- **Persistent database** – relational store for users, goals, requests, lobbies, messages, ratings, reports, blocks and OTP tokens.
- **Real-time layer** – Socket.IO for in-lobby chat and the private notes-request chat (planned).

- **Authentication** – JWT sessions plus college-email OTP verification, delivered by an SMTP relay (Brevo).
- **File storage** – Cloudinary/Firebase, for notes uploads and profile photos (planned).
- **Deployment** – Vercel (frontend) and Render/Railway (backend), for a shareable live demo.

2.2 System Design

2.2.1 Use Case Diagram

The use case model identifies three stakeholders – Student, Admin and Mail Service. Students sign up and log in (both including email OTP verification, delivered by the Mail Service), post and browse goals, request to join a goal, and – if accepted – coordinate in a lobby chat. Students can also cancel a goal they posted, share notes and resources, rate completion or flag a no-show, and report or block another user. Admins review reports raised against users. Figure 2.1 shows the full model.

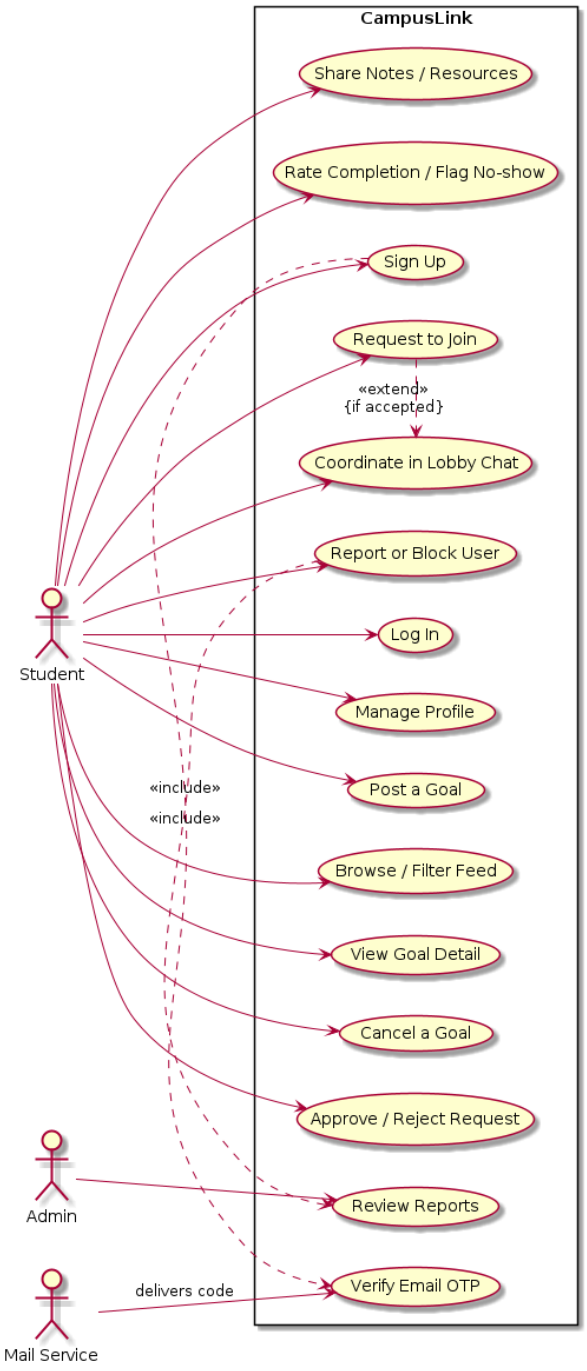

Figure 2.1: Use Case Diagram of CampusLink

2.2.2 Sequence Diagram

Figure 2.2 traces a single goal-matching interaction: the requester opens the feed, the frontend asks the backend for goals, the backend reads them from the database and returns them for display. When the requester sends a join request, the backend notifies the organizer, and once the organizer approves, the requester is accepted and added to the lobby.

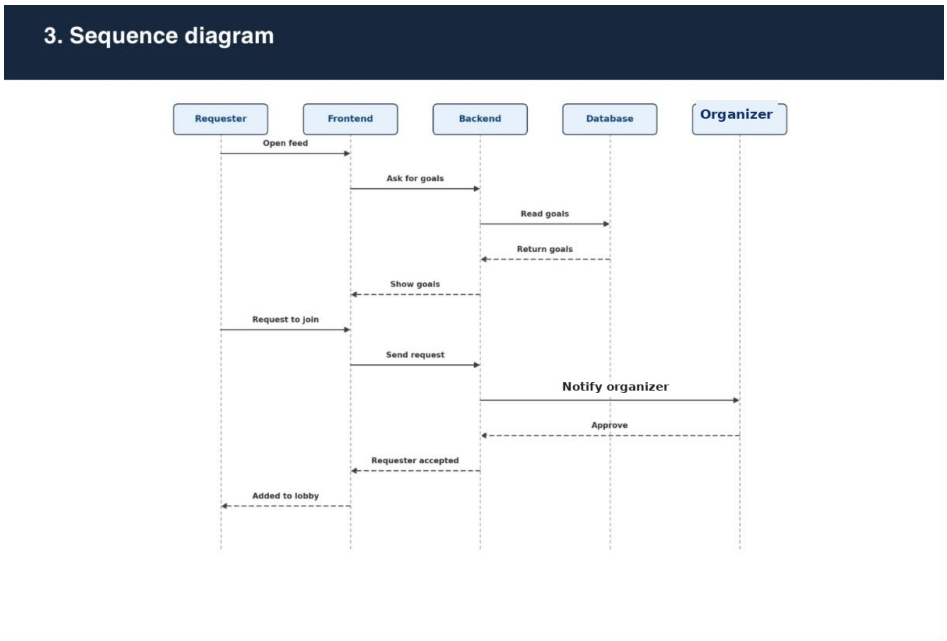

Figure 2.2: Sequence Diagram for a CampusLink Goal Request

2.2.3 Activity Diagram

Figure 2.3 follows the full lifecycle of a goal across the Organizer, Requester and System swimlanes: a goal is posted in the `OPEN` state, discovered on the feed, and requested by an interested student. If auto-accept is enabled the system accepts automatically; otherwise the organizer reviews and approves or rejects the request. Each accepted request increments the goal's accepted count; once the headcount is reached the goal is marked `FULL` and its lobby `CONFIRMED`, otherwise it stays `OPEN`. Both sides then coordinate in the lobby chat, and once the activity happens, the system records whether everyone showed up – adjusting reliability scores up or down accordingly – before marking the goal `COMPLETED`.

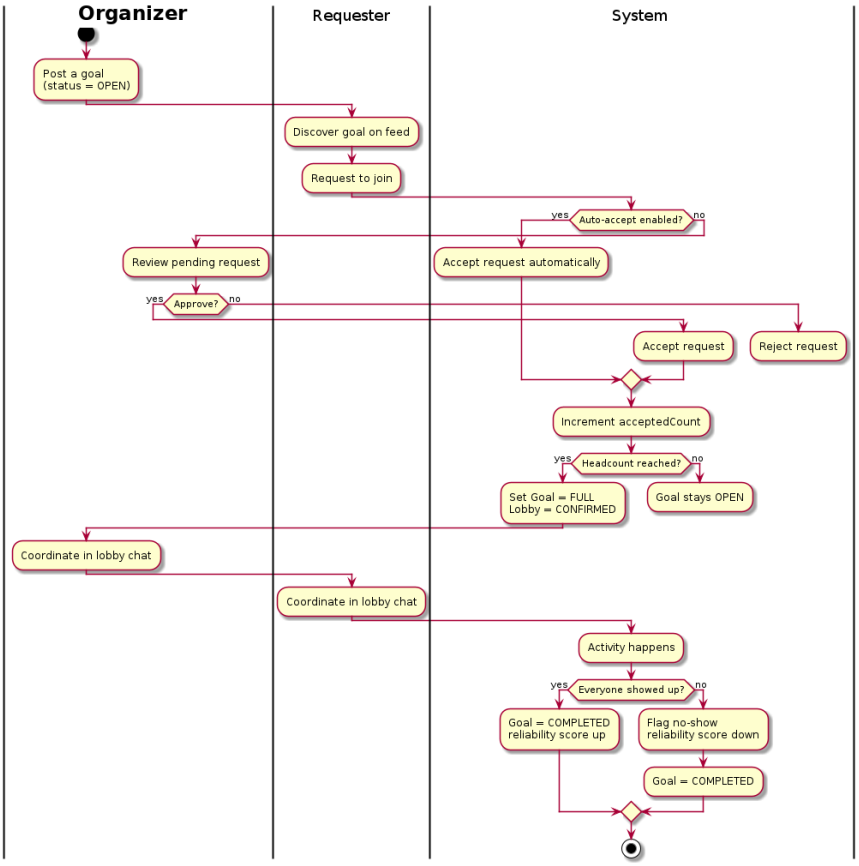

Figure 2.3: Activity Diagram of the Goal Lifecycle

2.3 Database and Domain Model Design

The domain model, shown in Figure 2.4, is organised around two service layers – `AuthService` and `GoalService` – and the entities they manage.

- **User** – id, email, name, verification flag, branch, year, gender, role, reliability score, goals-completed count and no-show count; exposes `getProfile()` and `updateProfile()`.
- **OtpToken** – id, email, code hash, purpose, expiry and attempt count, managed by `AuthService` for signup and login verification.
- **Goal** – id, category, title, description, date/time, headcount, accepted count, anonymous flag, gender preference, auto-accept flag, status and expiry; exposes `cancel()` and `isFull()`, and is managed end-to-end by `GoalService`.
- **Request** *(planned)* – id, status and creation time, linking a user to a goal, with `accept()` and `reject()` operations.
- **Lobby** *(planned)* – id and status, formed once a goal is confirmed, containing lobby members and messages.
- **LobbyMember** *(planned)* – user id and joined-at timestamp, linking users to a lobby.
- **Message** *(planned)* – body and sent-at timestamp, exchanged within a lobby.
- **Rating** *(planned)* – score and no-show flag, given by one user to another after a goal closes.
- **Report** *(planned)* – reason and status, filed by one user against another.
- **Block** *(planned)* – creation timestamp, recording that one user has blocked another.

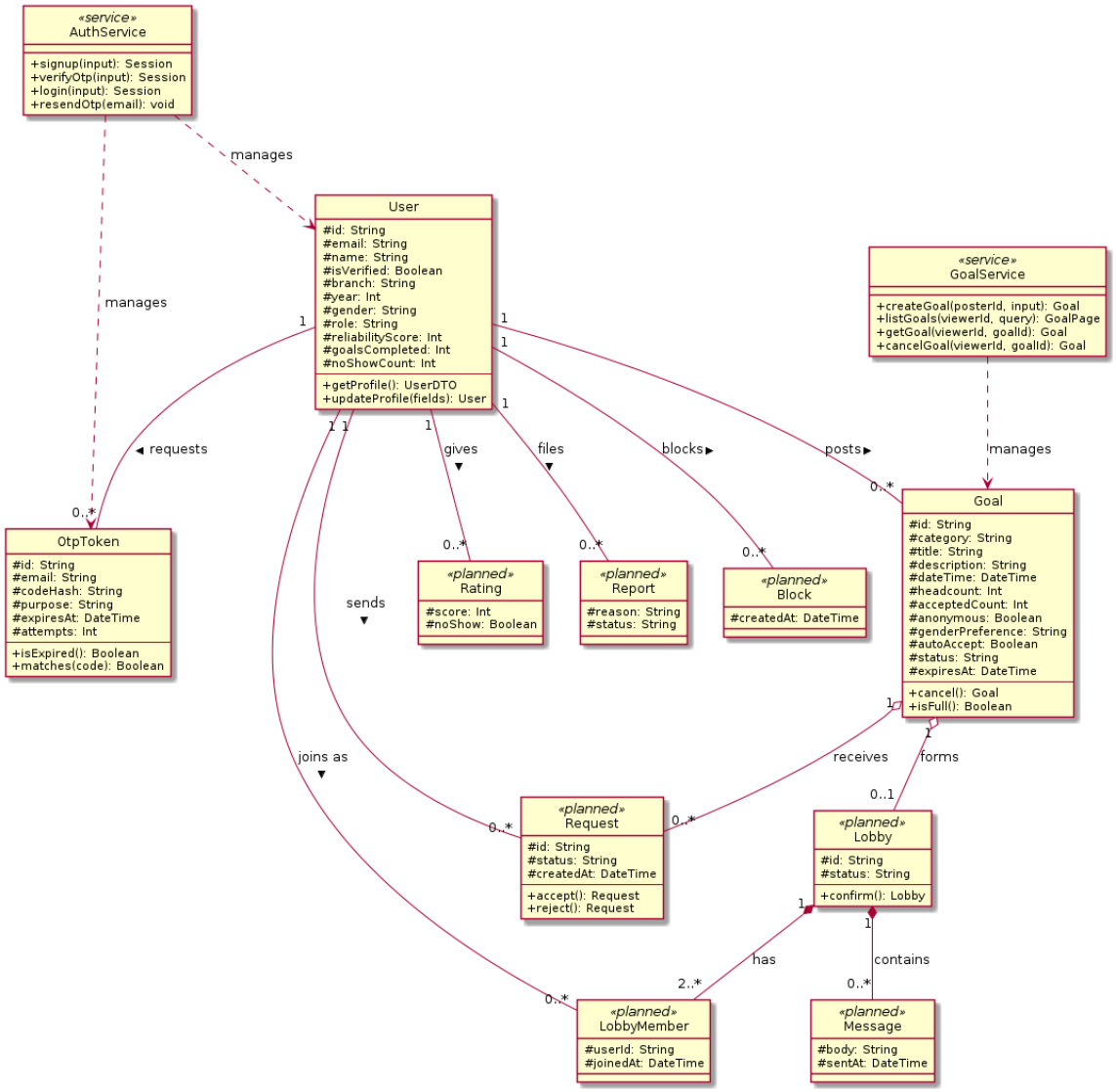

Figure 2.4: Domain / Class Diagram of CampusLink (entities marked *planned* are designed but not yet implemented in the current build)

2.4 Software Design

CampusLink is organised as a service-oriented web application in which the frontend communicates with backend services through a REST API. `AuthService` owns signup, OTP verification, login and OTP resend; `GoalService` owns goal creation, listing (with viewer-aware filtering), retrieval and cancellation. Keeping these as separate services – rather than one large controller – means the request/lobby state machine and the trust-and-safety logic (Section 2.5.4) can be added onto `GoalService` without touching authentication, and vice versa.

2.5 Implementation Details

2.5.1 Authentication and User Access

Access to the platform is restricted to verified students. Signup and login both go through a college-email OTP step before a session is issued, matching the "Verify Email OTP" use case in Figure 2.1. Figure 2.5 shows the live sign-in screen ("Welcome back – sign in to continue to CampusLink") and the marketing landing page students see before authenticating.

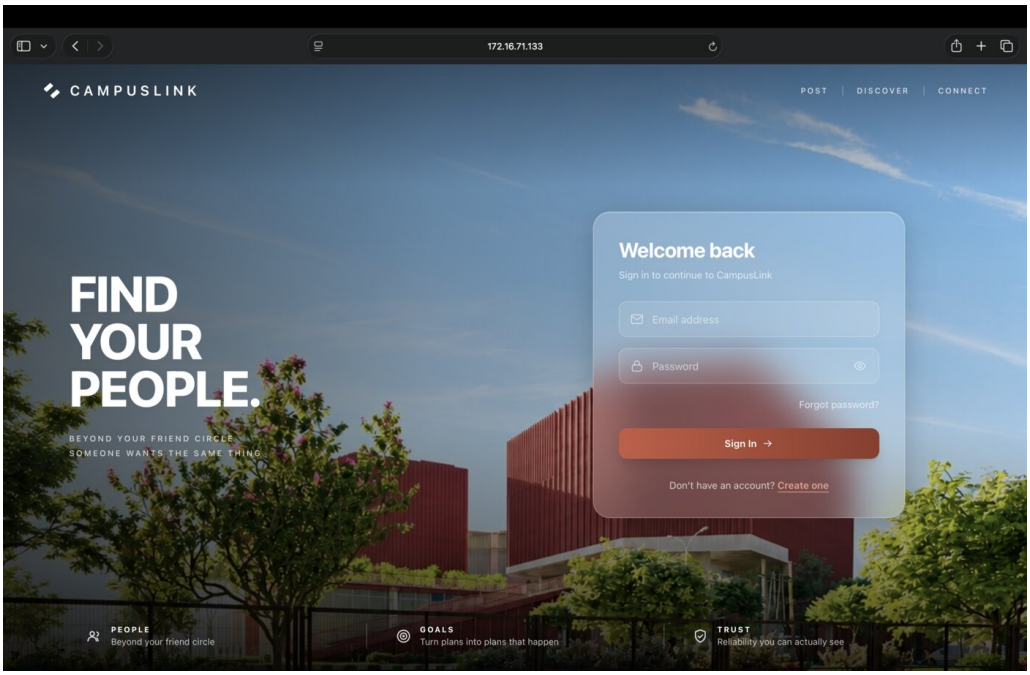

Figure 2.5: CampusLink landing and sign-in screen

2.5.2 Goal Posting and Discovery

Once signed in, a student lands on the feed (Figure 2.6), which lists goals under category filters – Sports, Gym, Study group, Travel/cab share, Food, Events, Notes/resources and Other – plus a personal "My goals" view. Each card on the feed shows only the category, title, description, date/time, remaining spots and the organizer's reliability score; identity is withheld unless the organizer chose to be identified, consistent with the anonymous-posting design in the project proposal.

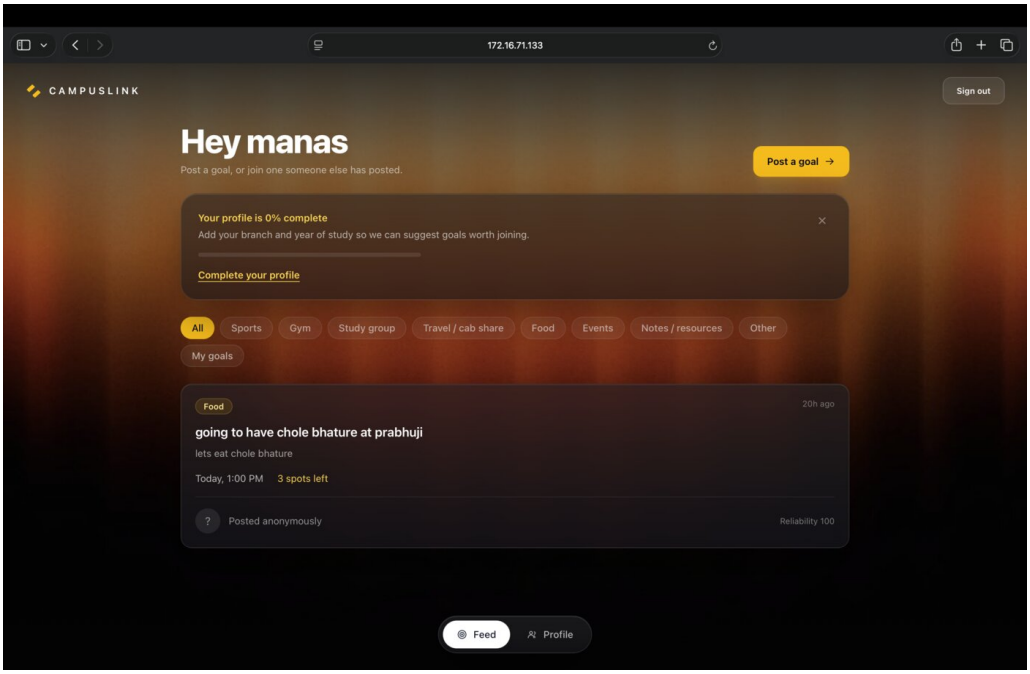

Figure 2.6: CampusLink feed, showing category filters and a live anonymous goal post

2.5.3 Profile and Reliability

The profile page (Figure 2.7) surfaces the fields that the trust-and-safety layer depends on: a verified badge, branch and year of study, a reliability score out of 100, and a count of goals completed. A profile that is missing branch or year is nudged towards completion, since these fields feed the rule-based match suggestions described in the project proposal.

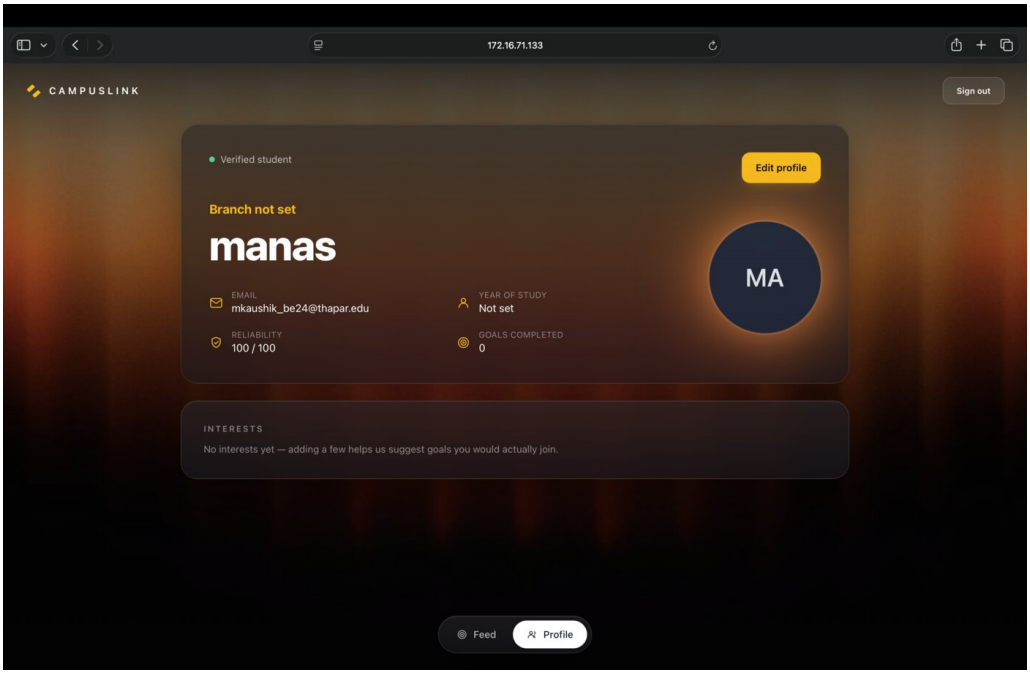

Figure 2.7: CampusLink student profile view

2.5.4 Request, Lobby and Trust-and-Safety Workflow

The request-to-join, approval and lobby-formation flow follows the activity diagram in Figure 2.3: a request is either auto-accepted or queued for the organizer's manual approval, and a lobby is confirmed once the goal's headcount is reached. The trust-and-safety layer around this loop – report/block, the gender-preference filter, and the pre-/post-acceptance visibility rule for anonymous posts – is designed per Figure 2.4 but, as noted in Section 2.1, remains to be implemented alongside the request and lobby entities.

2.5.5 Current Build Status

A local development run of the project confirms that the authentication and feed loop already works end-to-end: the Vite frontend serves on port 5173, the Express API listens on port 4000 with a passing health check, and the mail transport used for OTP delivery connects successfully to its SMTP relay. Server logs from that session show a successful login (`POST /api/auth/login 200`) followed by successful goal and profile fetches (`GET /api/goals 200`, `GET /api/users/me 200`), which is the same request path modelled in Figure 2.2.

Chapter 3

Results and Evaluation

3.1 Testing Methodology

Testing is organised around the end-to-end goal-matching loop: authentication, feed discovery, goal posting, the request/approval/lobby state machine, trust-and-safety actions, and post-goal rating.

- Functional testing of college-email OTP verification, signup and login.
- Feed retrieval, category/date filtering and goal-detail display.
- Goal posting, including the auto-accept and anonymous-posting options.
- Request-to-join, manual approval/rejection, and auto-accept branches.
- Lobby formation once headcount is reached, and in-lobby chat delivery.
- Trust-and-safety actions: report, block, gender-preference filtering.
- Goal-completion confirmation, no-show flagging and rating capture.

3.2 Functional Verification

Selecting a goal from the feed should lead to its detail view; requesting to join should lead to either auto-acceptance or a pending approval; and a headcount being reached should confirm the lobby. These flows correspond directly to the supplied use case, sequence and activity diagrams (Figures 2.1–2.3).

3.3 Evaluation Criteria

Primary metric

Lobby fulfilment rate – the percentage of posted goals that reach their required headcount within a reasonable window (the goal's own date/time, or 48 hours for undated ones). This is the metric that most directly reflects whether the core loop works.

Secondary metrics

- **Time-to-first-request** – how long a goal sits on the feed before someone requests to join; a proxy for whether discovery and filtering are working.

Flow	Verified behaviour	Result
Signup & OTP verification	Registering with a college email triggers an OTP email through the configured SMTP relay and issues a session on success.	Pass
Login	Submitting valid credentials returns an authenticated session and loads the profile and feed.	Pass
Feed retrieval	Opening the feed requests goals from the backend and renders category, timing and spots-left metadata.	Pass
Goal request & lobby formation	A join request follows the auto-accept/approve branch and forms a lobby once headcount is reached.	Design-verified
Trust & safety actions	Anonymous posting hides identity on the feed; report/block and gender filtering restrict interaction as designed.	Design-verified
Rating & no-show flagging	Goal completion prompts confirmation, adjusting the reliability score accordingly.	Design-verified

Table 3.1: Summary of functional verification against the design diagrams. "Pass" reflects behaviour observed in the running development build; "Design-verified" reflects flows confirmed against the use case, sequence and activity diagrams but not yet exercised in the current codebase.

- **No-show rate** – the percentage of confirmed lobbies where at least one member does not show up, indicating whether the reliability-scoring layer has any effect over time.
- **Resource-request response rate** – the percentage of notes/resource requests that receive at least one private response.
- **User-reported clarity/safety** – a short 1–5 feedback question after using the anonymous or gender-filter features, to check whether people actually feel safer using them.

Pilot validation

An internal pilot with 15–20 students from within the team's own batch and hostel block is planned, mixing genuine goal posts with a few seeded ones so the request/lobby flow has enough activity to evaluate – since a cold feed with no users would not exercise the matching or trust layer at all.

3.4 Analysis

The proposed evaluation emphasises the parts of CampusLink that a simple completion count cannot capture. A reliability score that only rewards completed goals looks identical for a consistently punctual user and one who repeatedly shows up late; folding in no-show flags gives a more honest signal. Similarly, time-to-first-request says more about whether the feed and filters are doing their job than a raw count of goals posted does. The trust-and-safety features (anonymous posting, gender filtering) are deliberately evaluated by a subjective safety

question rather than only by usage counts, since a feature nobody trusts enough to use would otherwise look identical, in the data, to a feature nobody needs.

Chapter 4

Conclusion

4.1 Summary of Work

CampusLink combines goal-based companion matching, a trust-and-safety layer and real-time coordination in a layered three-tier web platform. The core workflow forms a continuous loop – post, discover, request, form a lobby, coordinate, close and rate – and the same loop is reused, with a private chat in place of a public lobby, for academic notes and resource sharing. The current build already carries students through authentication, profile management and feed discovery; the request/lobby state machine and the trust-and-safety actions are fully specified in the design (Figures 2.1–2.4) as the next slice of work.

4.2 Key Findings

- A public feed combined with a request/approve step turns an unstructured, WhatsApp-group problem into an auditable, trackable workflow.
- Reliability scoring and no-show flagging give a requester a trust signal about a stranger before they commit time to joining that person's plan.
- Anonymous posting only works safely with a pre-/post-acceptance boundary: identity stays hidden on the public feed but is revealed to a requester once accepted into the lobby.
- Reusing the same post/request/fulfil state machine for notes-sharing avoids building and maintaining a second workflow for what is functionally the same problem.
- Keeping the matching "intelligence" rule-based – shared branch, interests and past categories – keeps the system explainable and scoped to what a single-semester course project can engineer properly.
- The authentication and feed loop are demonstrably functional in the current build; the request, lobby, chat and trust-and-safety layers remain to be implemented (Section 4.3).

4.3 Future Work and Recommendations

1. Implement the request/lobby state machine shown in Figure 2.3, including auto-accept, manual approval/rejection and headcount tracking.
2. Build the Socket.IO real-time layer for in-lobby chat and the private notes-request chat.
3. Implement report/block, the gender-preference filter, and the anonymous pre-/post-acceptance visibility rule described in Section 2.5.4.

4. Add no-show flagging and the post-goal rating flow that feeds the reliability score already shown on the profile page.
5. Build the rule-based match-suggestion engine (shared branch, interests, past categories) and hostel/proximity-based matching.
6. Finalise the relational schema shown in Figure 2.4 against the implemented database migrations.
7. Add continuous integration (build and lint on every push) and deploy the frontend and backend for a live, shareable demo.
8. Run the planned internal pilot (15–20 students) to measure lobby fulfilment rate and no-show rate against real usage.

4.4 Final Remarks

CampusLink is built around the idea that campus coordination should not depend on already knowing the right people. By pairing a simple, auditable post/request/lobby loop with a trust-and-safety layer built in from the start – rather than added on afterwards – the platform aims to make stranger-to-stranger coordination on campus feel workable rather than risky. The working authentication and feed loop shown in this report is the foundation that the request/lobby, chat and trust-and-safety layers are designed to build on next.